

LAB 1 - DELIVERABLES

Sequential Multiplier Using Repeated Addition

Name: R SARANG

Roll Number: EC23I2015

Course: VLSI Testing Practice (EC5034)

30th August 2026

Contents

1	Algorithm	2
2	Datapath Elements	3
3	Datapath Block Diagram	4
4	Control Signals	4
5	Status Signals	5
6	Timing Diagram	6
7	FSM Design	7
8	Integrate & Verify	8
8.1	Verilog RTL	8
8.1.1	Datapath (<code>datapath.v</code>)	8
8.1.2	Controller (<code>controller.v</code>)	9
8.1.3	Top Module (<code>top.v</code>)	10
8.2	Testbench (<code>tb_top.v</code>)	11
8.3	Test-Case Results	13
8.4	Cycle-Count Verification	13
8.5	3×12 vs 12×3 Observation	14

Aim

To design and verify a 4-bit unsigned sequential multiplier using repeated addition, consisting of one adder and an FSM-based control path.

1 Algorithm

The sequential multiplier computes the product of two 4-bit unsigned numbers (multiplicand A and multiplier B) using repeated addition. The algorithm initializes an 8-bit accumulator to zero. In each computation step, it adds the multiplicand to the accumulator and decrements the multiplier by 1. This process is repeated until the multiplier reaches zero.

Pseudocode:

Step 1: Wait for start to become high.

Step 2: When start is high:

- Reset accumulated product register to zero.
- Load multiplicand-in into the 4 bit Working Multiplicand Register.
- Load the multiplier-in into the 4 bit loop counter.

Step 3: Repeated Addition:

- Until the loop counter > 0 , do the following every clock cycle:
 - Add the value in the Working Multiplicand Register to the current value in the Accumulated Product Register.
 - Decrement loop counter by 1

Step 4: When loop counter reaches zero:

- Stop adding
- Set the mult-complete output signal to HIGH
- Answer is what is present in Accumulated Product Register.

Figure 1: Algorithm Pseudocode

2 Datapath Elements

Datapath Element	Usage	Width
Working Multiplicand Register	Storing the i/p multiplicand-in.	4-bit
Loop Counter	Down counter that stores multiplier-in	4-bit
Accumulated Product Register	Storing the running sum & ultimately the required O/p.	8-bit
Adder	Adds the multiplicand to the accumulated product	8-bit
Zero Detector	Checking if the loop counter has reached zero	1-bit

Figure 2: Datapath Element Usage

- **Accumulated Product Register:** 8-bit register to hold the running sum.
- **Working Multiplicand Register:** 4-bit register to store operand A.
- **Loop Counter:** 4-bit down-counter to store operand B and decrement each cycle.
- **Adder:** 8-bit adder to add the working multiplicand to the accumulated product.
- **Zero Detector:** Combinational logic to check if the loop counter is zero.

3 Datapath Block Diagram

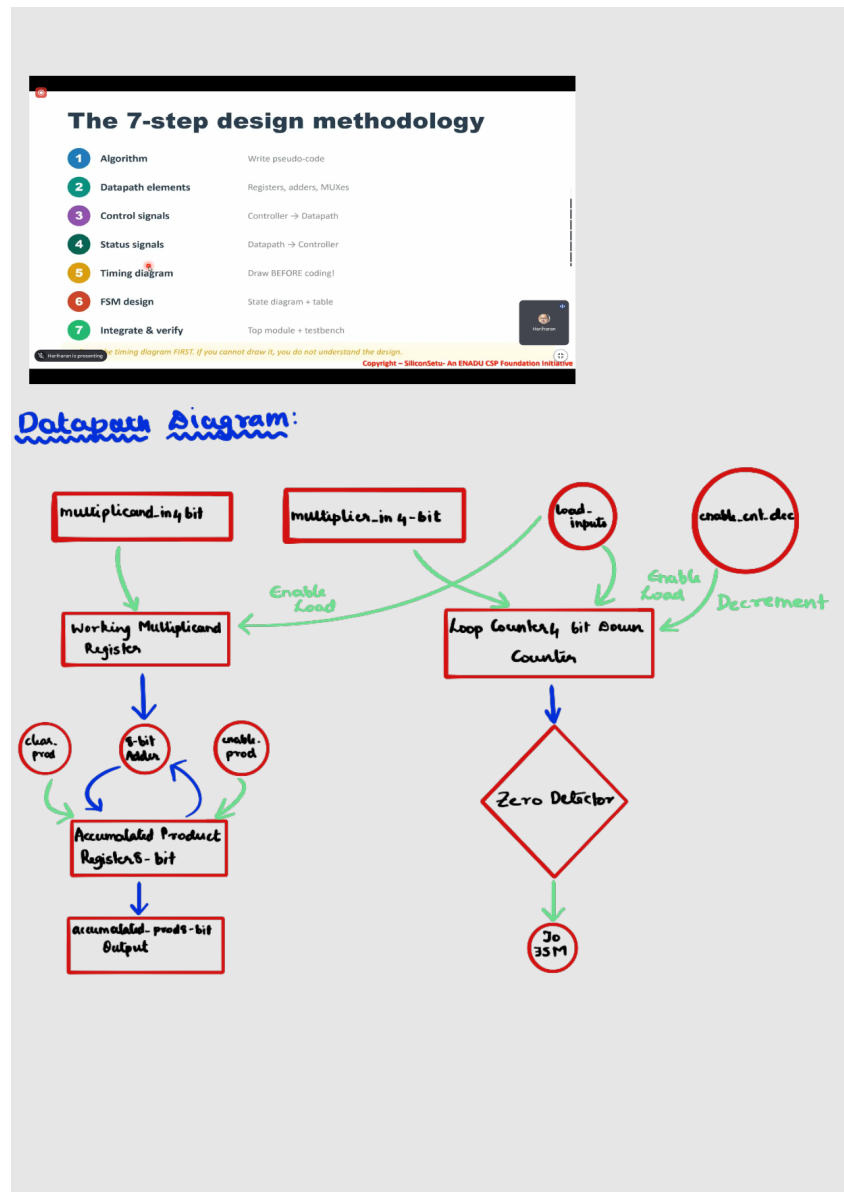


Figure 3: Complete Datapath Block Diagram

4 Control Signals

Control signals are generated by the FSM (Controller) and used to orchestrate the operations in the Datapath.

Signal	Direction	Function
load_inputs	Controller → Datapath	Loads the input operands (A into multiplicand reg, B into counter).
clear_prod	Controller → Datapath	Clears the 8-bit product register (accumulator) to 0.
enable_prod	Controller → Datapath	Enables product update (addition of multiplicand to accumulator).
enable_cnt_dec	Controller → Datapath	Enables loop counter operation (decrements the counter by 1).
mult_complete	Controller → Output	Indicates completion of the multiplication.

5 Status Signals

Status signals are generated by the Datapath and provided to the FSM (Controller) to make transition decisions.

Signal	Direction	Function
loops_zero_flag	Datapath → Controller	Indicates that the counter has reached zero.

6 Timing Diagram

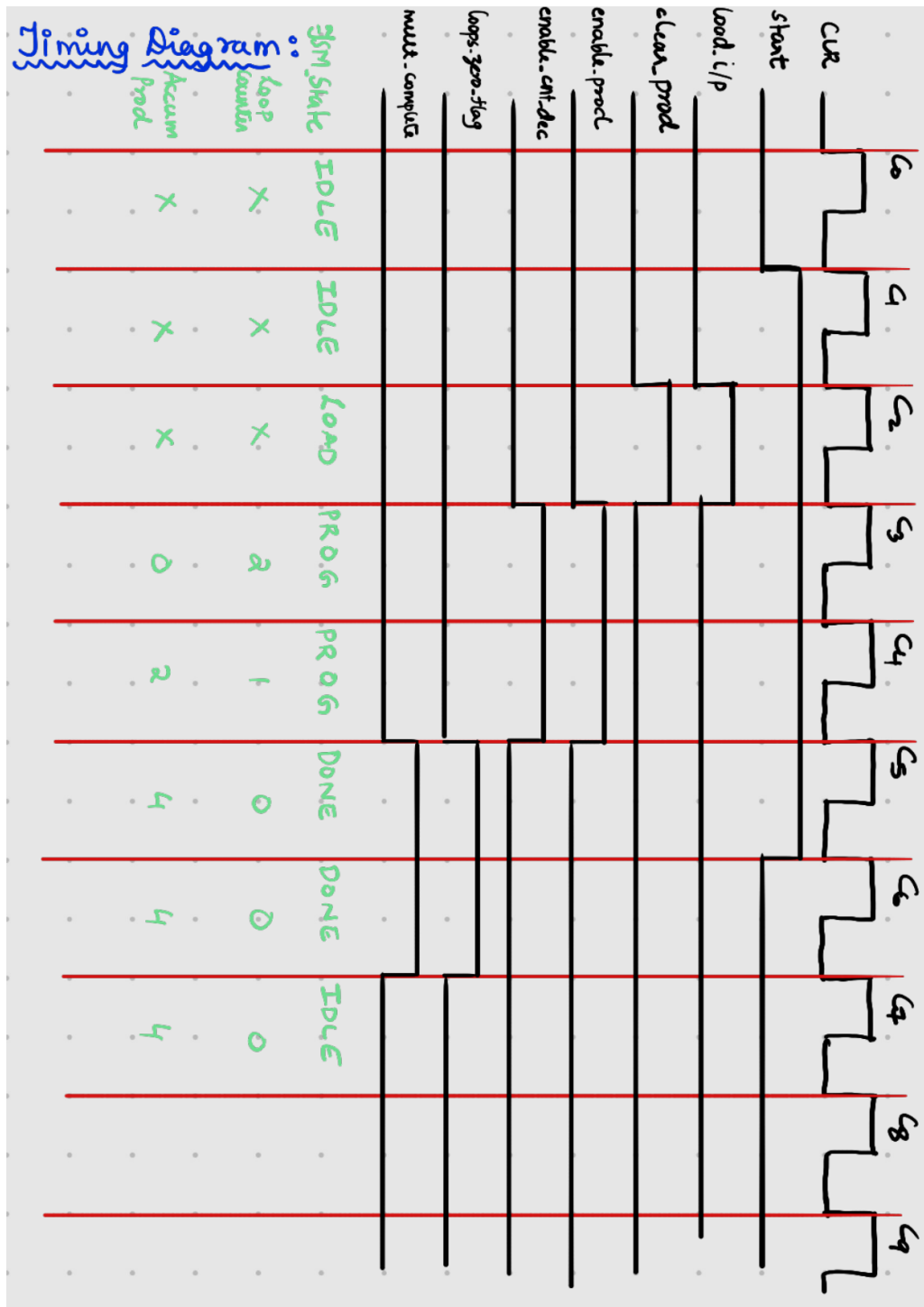


Figure 4: Timing Diagram for Control and Data Signals

7 FSM Design

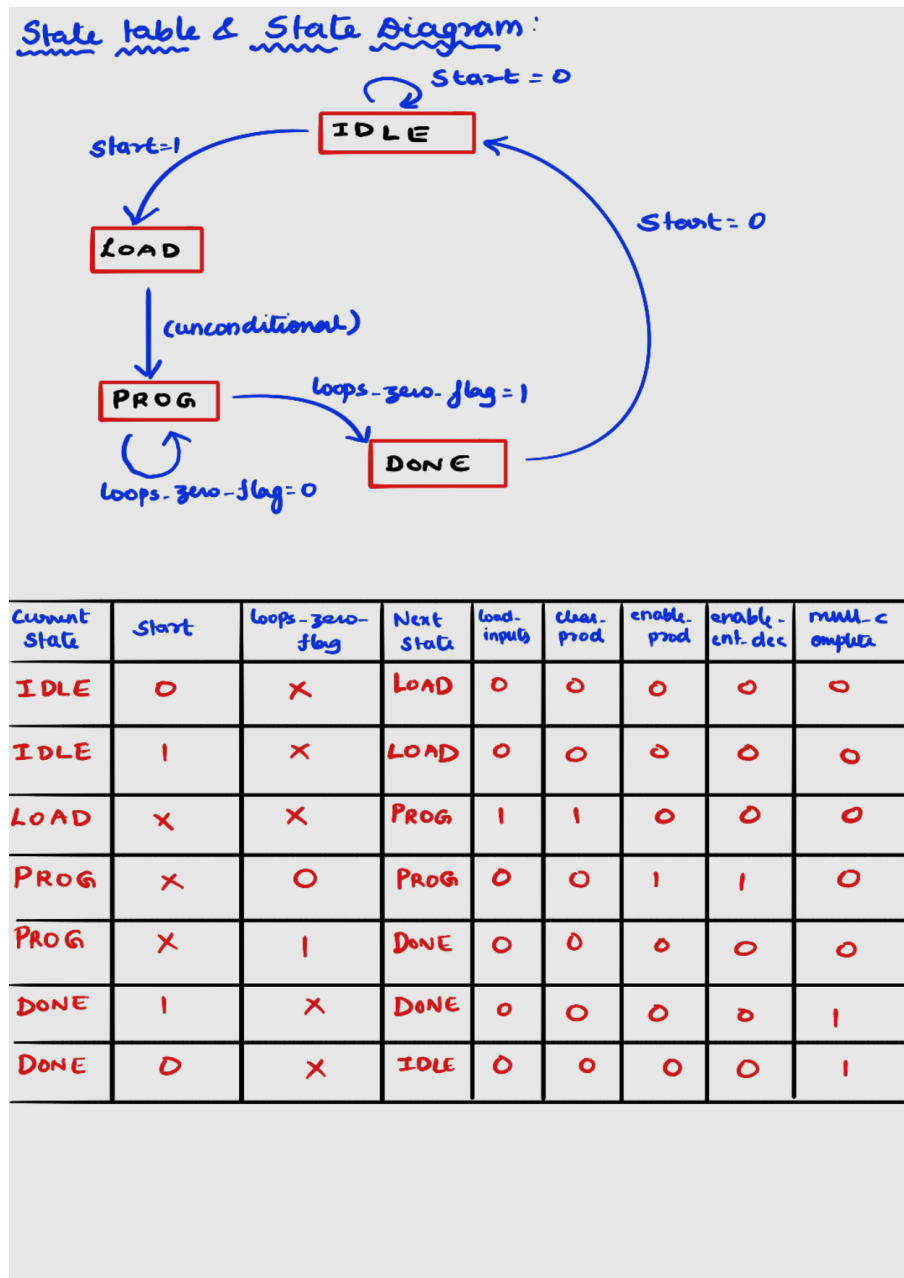


Figure 5: FSM State Table and State Diagram

Current State	start	loops_zero_flag	Next State	Outputs Asserted
IDLE (00)	0	X	IDLE (00)	None
IDLE (00)	1	X	LOAD (01)	None
LOAD (01)	X	X	PROG (10)	load_inputs=1, clear_prod=1
PROG (10)	X	0	PROG (10)	enable_prod=1, enable_cnt_dec=1
PROG (10)	X	1	DONE (11)	None
DONE (11)	1	X	DONE (11)	mult_complete=1
DONE (11)	0	X	IDLE (00)	mult_complete=1

8 Integrate & Verify

8.1 Verilog RTL

The Verilog RTL design consists of the datapath, controller, and top-level modules.

8.1.1 Datapath (datapath.v)

```
1 module datapath (  
2     input clk,  
3  
4     // Inputs  
5     input [3:0] multiplicand_in,  
6     input [3:0] multiplier_in,  
7  
8     // Control Signals  
9     input load_inputs,  
10    input clear_prod,  
11    input enable_prod,  
12    input enable_cnt_dec,  
13  
14    // Status Signal  
15    output loops_zero_flag,  
16  
17    // Data Output  
18    output reg [7:0] accumulated_prod  
19 );  
20  
21    // Internal Registers  
22    reg [3:0] working_multiplicand;  
23    reg [3:0] loop_counter;  
24  
25    // APR (8-bit)  
26    // EC23I2015  
27    always @(posedge clk) begin  
28        if (clear_prod == 1'b1) begin  
29            accumulated_prod <= 8'd0;  
30        end  
31        else if (enable_prod == 1'b1) begin  
32            accumulated_prod <= accumulated_prod + working_multiplicand;  
33        end  
34    end  
35  
36    // Working Multiplicand Register (4-bit)  
37    always @(posedge clk) begin  
38        if (load_inputs == 1'b1) begin  
39            working_multiplicand <= multiplicand_in;  
40        end  
41    end  
42  
43    //4 bit Down Loop Counter  
44    always @(posedge clk) begin  
45        if (load_inputs == 1'b1) begin  
46            loop_counter <= multiplier_in;  
47        end  
48        else if (enable_cnt_dec == 1'b1) begin  
49            loop_counter <= loop_counter - 1; // Or loop_counter - 4'd1
```



```

50     end
51 end
52
53 // Zero Detector
54 assign loops_zero_flag = (loop_counter == 4'd0);
55
56 endmodule

```

8.1.2 Controller (controller.v)

```

1 module controller (
2     input clk,
3     input reset,
4
5     // Going To FSM
6     input start,
7     input loops_zero_flag,
8
9     // Coming from FSM (Control signals for datapath)
10    output reg load_inputs,
11    output reg clear_prod,
12    output reg enable_prod,
13    output reg enable_cnt_dec,
14
15    // Output to outside world
16    output reg mult_complete
17);
18
19 // Encodings for the required states
20 parameter IDLE    = 2'b00;
21 parameter LOAD    = 2'b01;
22 parameter PROG    = 2'b10;
23 parameter DONE    = 2'b11;
24
25 // State Registers
26 reg [1:0] current_state, next_state;
27
28 // State Memory
29 always @(posedge clk) begin
30
31     if (reset == 1'b1) begin
32         current_state <= IDLE;
33     end
34     else begin
35         current_state <= next_state;
36     end
37
38 end
39
40 // Next State & Output Logic
41 // EC23I2015
42 always @(*) begin
43     // Default outputs
44     load_inputs    = 1'b0;
45     clear_prod     = 1'b0;
46     enable_prod    = 1'b0;
47     enable_cnt_dec = 1'b0;
48     mult_complete  = 1'b0;

```

```

49     next_state      = current_state;
50
51     case (current_state)
52         IDLE: begin
53
54             if (start == 1'b1) begin
55                 next_state = LOAD;
56             end
57         end
58
59         LOAD: begin
60             load_inputs = 1'b1;
61             clear_prod = 1'b1;
62             next_state = PROG;
63
64         end
65
66         PROG: begin
67             if (loops_zero_flag == 1'b1) begin
68                 next_state = DONE;
69                 enable_prod = 1'b0;
70                 enable_cnt_dec = 1'b0;
71             end
72             else begin
73                 next_state = PROG;
74                 enable_prod = 1'b1;
75                 enable_cnt_dec = 1'b1;
76             end
77         end
78
79         DONE: begin
80             mult_complete = 1'b1;
81             if (start==1'b1) begin
82                 next_state=DONE;
83             end
84             else begin
85                 next_state=IDLE;
86             end
87         end
88     endcase
89 end
90
91 endmodule

```

8.1.3 Top Module (top.v)

```

1 module top (
2     input clk,
3     input reset,
4     input start,
5     input [3:0] multiplicand_in,
6     input [3:0] multiplier_in,
7
8     output mult_complete,
9     output [7:0] accumulated_prod
10 );
11
12 // Internal Wires connecting Controller and Datapath

```

```

13  wire load_inputs;
14  wire clear_prod;
15  wire enable_prod;
16  wire enable_cnt_dec;
17  wire loops_zero_flag;
18
19  // Instantiation for the controller
20  // EC23I2015
21  controller u_controller (
22      .clk          (clk),
23      .reset        (reset),
24      .start        (start),
25      .loops_zero_flag (loops_zero_flag),
26      .load_inputs   (load_inputs),
27      .clear_prod    (clear_prod),
28      .enable_prod    (enable_prod),
29      .enable_cnt_dec (enable_cnt_dec),
30      .mult_complete  (mult_complete)
31  );
32
33  // Instantiation for the datapath
34  datapath u_datapath (
35      .clk          (clk),
36      .multiplicand_in  (multiplicand_in),
37      .multiplier_in   (multiplier_in),
38      .load_inputs     (load_inputs),
39      .clear_prod      (clear_prod),
40      .enable_prod      (enable_prod),
41      .enable_cnt_dec   (enable_cnt_dec),
42      .loops_zero_flag  (loops_zero_flag),
43      .accumulated_prod (accumulated_prod)
44  );
45
46  endmodule

```

8.2 Testbench (tb_top.v)

```

1  `timescale 1ns / 1ps
2
3  module tb_top;
4
5      // Testbench Inputs
6      reg clk;
7      reg reset;
8      reg start;
9      reg [3:0] multiplicand_in;
10     reg [3:0] multiplier_in;
11
12     // Testbench Outputs
13     wire mult_complete;
14     wire [7:0] accumulated_prod;
15
16     // Instantiate Unit Under Test (UUT)
17     top uut (
18         .clk(clk),
19         .reset(reset),
20         .start(start),
21         .multiplicand_in(multiplicand_in),

```

```

22     .multiplier_in(multiplier_in),
23     .mult_complete(mult_complete),
24     .accumulated_prod(accumulated_prod)
25 );
26
27 // Clock Generation
28 initial begin
29     clk = 0;
30     forever #5 clk = ~clk; // 10ns Clock Period
31 end
32
33 // Test variables
34 integer cycle_count;
35 integer expected_product;
36 integer expected_cycles;
37
38 // Synchronous Test Task
39 // EC23I2015
40 task run_test(input [3:0] A, input [3:0] B);
41     begin
42         // Set inputs on falling edge of clock
43         @(negedge clk);
44         multiplicand_in = A;
45         multiplier_in    = B;
46         expected_product = A * B;
47         expected_cycles  = B + 2; // B COMPUTE cycles + 1 LOAD + 1 DONE
48         cycle_count      = 0;
49
50         // Assert start signal
51         start = 1'b1;
52
53         // Wait 1 clock cycle for LOAD phase
54         @(negedge clk);
55         start = 1'b0; // De-assert start
56
57         // Count cycles while waiting for completion
58         while (!mult_complete) begin
59             @(negedge clk);
60             cycle_count = cycle_count + 1;
61         end
62
63         // Display self-checking results
64         $write("Test: %2d x %2d | Exp Prod: %3d, Act Prod: %3d | Exp Cycles: %2d,
→ Act Cycles: %2d | ",
65             A, B, expected_product, accumulated_prod, expected_cycles,
→ cycle_count);
66
67         if (accumulated_prod == expected_product && cycle_count == expected_cycles
→ ) begin
68             $display("[ PASS ]");
69         end else begin
70             $display("[ FAIL ]");
71         end
72
73         // Wait 2 clock cycles before starting next test (Synchronous delay)
74         repeat (2) @(negedge clk);
75     end
76 endtask

```

```

77
78 // Test Execution Sequence
79 initial begin
80     // Initialize Inputs
81     reset = 1'b1;
82     start = 1'b0;
83     multiplicand_in = 4'd0;
84     multiplier_in    = 4'd0;
85
86     // Hold reset for 2 clock cycles
87     repeat (2) @(negedge clk);
88     reset = 1'b0;
89     repeat (2) @(negedge clk);
90
91     run_test(4'd0, 4'd0);
92     run_test(4'd0, 4'd5);
93     run_test(4'd5, 4'd0);
94     run_test(4'd1, 4'd1);
95     run_test(4'd1, 4'd15);
96     run_test(4'd15, 4'd1);
97
98     run_test(4'd3, 4'd4);
99     run_test(4'd15, 4'd15);
100
101     run_test(4'd3, 4'd12);
102     run_test(4'd12, 4'd3);
103
104     // Wait 2 clock cycles then finish simulation
105     repeat (2) @(negedge clk);
106     $finish;
107 end
108
109 endmodule

```

8.3 Test-Case Results

Test Case	A	B	Expected Product	Actual Product	Result
Corner (0x0)	0	0	0	0	PASS
Corner (0xN)	0	5	0	0	PASS
Corner (Nx0)	7	0	0	0	PASS
Corner (1x1)	1	1	1	1	PASS
Corner (1x15)	1	15	15	15	PASS
Corner (15x1)	15	1	15	15	PASS
Maximum Case	15	15	225	225	PASS
Commutativity 1	3	12	36	36	PASS
Commutativity 2	12	3	36	36	PASS

8.4 Cycle-Count Verification

Expected total cycles: $B + 2$ (Consists of 1 LOAD cycle + B COMPUTE cycles + 1 DONE cycle)

A	B	Expected Product	Expected Cycles	Actual Cycles	Result
0	0	0	2	2	PASS
3	4	12	6	6	PASS
3	12	36	14	14	PASS
12	3	36	5	5	PASS
15	15	225	17	17	PASS

8.5 3×12 vs 12×3 Observation

Both 3×12 and 12×3 result in the same product (36) due to the commutative property of multiplication. However, their execution times differ significantly in a sequential multiplier implemented using repeated addition.

- For 3×12 ($A = 3, B = 12$), the multiplier adds 3 to the accumulator twelve times. This requires 12 COMPUTE cycles, resulting in a total of 14 cycles ($12 + 2$).
- For 12×3 ($A = 12, B = 3$), the multiplier adds 12 to the accumulator three times. This requires only 3 COMPUTE cycles, resulting in a total of 5 cycles ($3 + 2$).

Conclusion: To minimize execution time in a repeated-addition sequential multiplier, the smaller operand should always be chosen as the multiplier (B).

Inference

The sequential multiplier was successfully designed by separating the logic into a datapath and a control path (FSM). The FSM effectively orchestrates the operations in the datapath, correctly managing the state transitions from IDLE to LOAD, to PROG (computation), and finally to DONE. Using a self-checking testbench, the design was validated across a comprehensive set of test cases including corner cases and worst-case scenarios, confirming its correctness and functional completeness.

Conclusion

In this lab, a 4-bit unsigned sequential multiplier was designed and verified using Verilog HDL. The separation of datapath and controller simplified the design process and debugging. Simulation results confirm that the FSM properly controls the repeated addition algorithm, producing accurate results for all test cases and corner conditions. The cycle-count analysis successfully demonstrated the performance dependency on the choice of the multiplier operand.